

CS-AML

Frontend Architecture & State Management Specification

Version 0.1

Purpose

Normative frontend architecture baseline for the CS-AML MVP. It defines frontend module boundaries, routing, state ownership, server-state synchronization, form and visualization state, authorization-aware rendering, concurrency handling, error recovery, accessibility, testing and frontend delivery conventions.

Core architecture axiom

Frontend state SHALL preserve canonical server truth, workflow versioning, permission boundaries and analytical uncertainty. The client SHALL NOT manufacture durable truth by promoting local, cached, inferred or visualization-only state into canonical state.

Status: Normative frontend implementation baseline for MVP 0.1

Dependencies: SRS · Technical Stack · UX · IA · Screen Inventory · Wireframe · UI Design System · High-Fidelity UI · Storybook

Document Control

Attribute	Value
Document ID	CSAML-FEARCH-0.1
Version	0.1
Status	Normative frontend architecture baseline
Primary audience	Frontend Engineer, UX Engineer, Tech Lead, QA, Security Reviewer
Reference stack	React 19 + TypeScript + Vite + Tailwind CSS 4 + shared Storybook design system
API assumption	Versioned REST API; canonical authorization and validation remain server-side
Normative verbs	SHALL / MUST / SHOULD / MAY

1. Scope and Non-Goals

This specification defines the frontend application architecture for the CS-AML MVP. It translates the approved screen and component specifications into implementation boundaries and state-management rules. It does not define backend business logic, canonical data schemas, authorization policy decisions, or API endpoint payloads; those are owned by backend specifications and the separate API Specification.

In scope	Out of scope
Feature-module organization, routing, data fetching, cache ownership, local state, forms, optimistic updates, concurrency, error handling, authorization-aware UI, visualization state, testing	Database model, server-side policy engine, evidence storage implementation, backend transaction rules
Frontend adaptation of canonical analytical semantics	New analytical semantics not present in framework/data model
Client security behavior and non-leakage rules	Treating UI hiding as an authorization control

Security boundary

Hiding a control or object in the UI is a usability measure, not an authorization decision. Every protected read or mutation SHALL remain authorized and validated by the API.

2. Architecture Principles

ID	Principle	Requirement
FE-P01	Server truth first	Canonical objects, permissions, versions, approvals and audit-relevant state originate from server responses.
FE-P02	State by ownership	Server state, form state, navigation state, workflow state, visualization state and ephemeral UI state SHALL have distinct owners.
FE-P03	URL is durable navigation state	Object IDs, case context, active tabs, filters and shareable search context SHOULD live in URL state where safe.
FE-P04	No monolithic global store	A single application-wide mutable store SHALL NOT become the default home for all state.
FE-P05	Permission-aware by construction	Components receive authorized data; restricted metadata SHALL not be cached or rendered merely because a route was guessed.
FE-P06	Version-aware editing	Editable canonical resources SHALL detect stale versions and prevent silent overwrite.
FE-P07	Derived views stay derived	Graph layouts, timeline positions and view filters SHALL NOT silently mutate canonical relationships, events or value flows.
FE-P08	Uncertainty preserved	Claim, fact, candidate, disputed, unknown and flow-class semantics persist through cache, form, visualization and export preview.
FE-P09	Recover locally	Component/feature failures SHOULD degrade locally before forcing a whole-app failure.
FE-P10	Accessible state	State changes, validation, loading and errors are exposed to keyboard and assistive technology.

3. Reference Frontend Stack

Concern	Reference	Rule
Runtime	React 19 + TypeScript	Strict TypeScript SHALL be enabled; `any` is exceptional and justified.
Build	Vite	Environment variables exposed to browser are explicitly allowlisted.
Styling	Tailwind CSS 4 + semantic CSS variables	Feature code consumes semantic tokens; raw status colors are prohibited where tokens exist.
Routing	React Router or equivalent	Routes use stable canonical IDs; case context is explicit.
Server state	TanStack Query or equivalent	Remote cache, invalidation, deduplication and mutation lifecycle are centralized here.
Forms	React Hook Form + schema validation or equivalent	Form state stays local to form boundary; server validation is mapped back to fields.
Schema types	Generated/OpenAPI-derived TypeScript types where feasible	Handwritten duplicate DTO types SHOULD be minimized.
Visualization	Cytoscape.js + approved timeline/value-flow primitives	Visualization state remains non-canonical.
Storybook	Storybook	Shared components and compound patterns require stories/test coverage.
Tests	Vitest/Jest + Testing Library + Playwright	Behavior first; implementation-detail tests discouraged.

4. Application Module Architecture

```
frontend/  
src/  
  app/  
    router/  
    providers/  
    error-boundaries/  
  features/  
    cases/  
    evidence/  
    entities/  
    relationships/  
  assets/  
  timeline/  
  valueflows/
```

typologies/
 hypotheses/
 assessments/
 search/
 graph/
 products/
 reviews/
 dissemination/
 administration/
 audit/
 shared/
 api/
 auth/
 components/
 design-system/
 hooks/
 types/
 utils/
 test/

Layer	May depend on	SHALL NOT depend on
app	features, shared	Feature internals via deep imports
feature	shared, explicitly published contracts from another feature	Another feature's private components/store internals
shared API/auth	generic infrastructure	Feature UI
design-system	tokens, generic utilities	Case/entity/business-specific behavior
screen composition	feature public components, design system	Backend implementation details

Boundary rule

Cross-feature imports SHOULD use explicit public barrels/contracts. If two features repeatedly depend on the same domain presentation primitive, promote that primitive deliberately rather than creating circular imports.

5. Routing and Navigation State

/cases
 /cases/:caseId/overview
 /cases/:caseId/evidence/:evidenceId
 /entities/:entityId?case=:caseId
 /analysis/value-flows?case=:caseId
 /products/:productId/review
 /search?q=...&type=entity&case=...

State	URL?	Reason
Canonical object ID	YES	Deep-linkable identity.
Case context	YES where relevant	Same entity may be opened inside different authorized case contexts.
Primary tab/section	YES	Back/forward navigation and shareable context.
Search query/facets	YES unless sensitive query policy forbids	Reproducibility and navigation.
Inspector open/closed	Usually NO	Ephemeral view state.
Unsaved form values	NO	Sensitive local state; do not leak into history/URL.
Graph viewport coordinates	Usually NO	Derived visualization state; MAY persist as named saved view later.
Protected-source identifiers	NO in ordinary route/query strings	Avoid browser history/log leakage.

6. State Taxonomy and Ownership

State class	Examples	Owner	Persistence
Server state	Case, entity, evidence metadata, assessment version, review status	Query/cache layer backed by API	Remote canonical; client cache ephemeral
Form state	Unsaved charter fields, hypothesis edit, relationship editor	Form boundary	Memory; optional safe draft policy only
Navigation state	Route, case context, active tab, filters	Router/URL	Browser history
Workflow UI state	Wizard step, review panel state	Feature component	Memory; derive from server when material
Visualization state	Selected graph nodes, zoom, hidden layers, timeline scale	Visualization controller	Memory; optional saved view, never canonical relationship truth
Ephemeral UI state	Modal open, tooltip, toast, drawer	Local component	Memory
Session/auth state	Authenticated principal, session expiry, coarse capabilities	Auth provider	Memory + IdP/session cookie/token model
Preference state	Density, reduced motion, column visibility	Preference service/local storage if approved	Local or server preference; no sensitive case content

Hard rule

Canonical server state SHALL NOT be copied into a global mutable store merely to make it easy to access. Use normalized/query cache access and feature selectors instead. Duplicate copies create version drift and authorization leakage risk.

7. Server-State Management

- Every query key SHALL include all dimensions that materially change authorization or representation, including canonical object ID, case context where relevant, and relevant filters.
- Cache entries SHALL be invalidated or updated after successful canonical mutations according to explicit mutation contracts.
- Unauthorized/forbidden responses SHALL not be cached as if they were ordinary empty data.
- Query retries SHOULD be disabled or limited for 401/403/404 non-disclosing authorization outcomes.
- Background refetch MAY refresh read screens; editable screens require version-aware reconciliation before replacing user-visible data.
- Prefetch MAY be used for ordinary objects but SHALL NOT prefetch protected-source identity or highly restricted content without an explicit authorized task.
- Client caches SHALL be cleared on logout/session revocation and SHOULD be scoped to the authenticated principal.

Query example	Key shape
Case overview	["case", caseId, "overview"]
Entity detail in case	["entity", entityId, {caseId}]
Evidence reader	["evidence", evidenceId, {caseId, version}]
Search	["search", {q,type,caseId,filters,page}]
Review product	["product", productId, {version, mode:"review"}]

8. Mutation and Concurrency Model

Mutation type	Optimistic?	Rule
Low-impact preference	YES	May update locally and rollback on failure.
Task status	Conditional	Only if authorization and version conflicts are simple; failure visibly rolls back.
Entity merge/unmerge	NO	High-impact/reversible canonical action; wait for server result.

Mutation type	Optimistic?	Rule
Evidence metadata affecting provenance	NO	Server validates integrity/version.
Assessment finalization	NO	Requires server gate/version validation.
Review approval/rejection	NO	High-impact decision and independence checks.
Dissemination approval/export	NO	Server authorization/audit must succeed first.
Graph selection/layout	Local only	Not a canonical mutation.

Concurrency contract

Editable canonical resources SHOULD use an explicit version token (for example record_version/ETag). On conflict, the frontend SHALL present stale-version recovery and SHALL NOT silently overwrite the newer server version.

9. Form Architecture

- Forms SHALL use client-side schema validation for immediate feedback and server-side validation as authority.
- Unknown, null, approximate and zero SHALL remain distinct values where the domain model distinguishes them.
- Unsaved changes SHALL trigger safe-navigation protection for material forms.
- Form submission SHALL preserve field-level server errors and non-field errors.
- High-impact forms SHALL include explicit review summary before final action where defined by UX.
- Dynamic controlled-vocabulary fields SHALL retain term IDs and display labels; labels alone are not canonical values.
- Protected-source identity SHALL not be auto-filled into ordinary evidence or export forms.

Form family	Draft behavior
Case/charter	Local unsaved state; server version after save.
Evidence upload	Upload session state; failed upload does not create completed evidence.
Entity/relationship	Local edit; canonical mutation on save.
Hypothesis/assessment	Local edit + optional explicit server draft version; no invisible autosave that changes audit-relevant content.
Product editor	Versioned draft; autosave MAY be used if server-side version/audit semantics are explicit.
Review/dissemination	No optimistic finalization; server response authoritative.

10. Authorization-Aware Rendering

Pattern	Requirement
Capability check	UI MAY use server-provided capabilities to show/disable actions.
Object access	No component SHALL infer access solely from role name; object/case policy matters.
Non-disclosing denial	A 404/403 policy response SHALL map to the approved non-leaking system state.
Protected source	Protected-source identity components SHALL exist in separately permissioned feature boundary and SHALL not be imported into ordinary evidence screens.
Export preview	Only server-authorized export candidates are rendered as selectable.
Counts/facets	Counts returned by API are treated as already permission-filtered; frontend SHALL not reconstruct hidden totals.

11. Error and Recovery Architecture

Failure class	UI behavior
Network/transient	Local retry with clear status; preserve safe unsaved state where feasible.
Authentication expired	Suspend protected work, re-authenticate; do not discard local form without warning where secure recovery is possible.
Authorization changed	Remove restricted cached content and transition to non-disclosing access state.
Validation	Field/non-field errors near source; no generic toast-only failure.
Concurrency conflict	Open compare/reload/resolve flow; never auto-force overwrite.
Integrity warning	Blocking error treatment for evidence integrity issue.
Partial feature failure	Local error boundary keeps surrounding case workspace usable.
Fatal application failure	Global recovery boundary with correlation ID; no sensitive payload in error display/log.

12. Visualization State Management

Domain	Canonical state	Frontend-only state
Graph	Entity/Relationship/Asset/ValueFlow objects	node positions, selection, zoom, hidden edge classes, inspector state
Timeline	Event records and temporal precision	scale, viewport, filters, expanded cards
Value Flow	ValueFlow/legs/class/evidence	canvas layout, selected leg, comparison overlays
Hypothesis matrix	Hypothesis/evidence link disposition	sorting, collapsed rows, temporary filter
Evidence reader	Evidence/Extract objects	page viewport, zoom, transient selection before extract save

Derived-state rule

Layout algorithms, centrality calculations, path highlighting and client-side clustering MAY assist navigation, but SHALL remain visibly derived and SHALL NOT create canonical relationships, risk labels or adverse assessment state.

13. Search and Filter State

- Global search query and approved facets SHOULD be URL-addressable.
- Facet counts are server-provided permission-filtered values.
- Debounced queries SHALL be cancellable to avoid displaying stale results after a newer query completes.
- Sensitive query strings MAY require history/logging safeguards; protected-source identity SHALL not be exposed through ordinary global search.
- Search results SHALL display canonical object type/status and context without manufacturing analytical priority from ranking position.

14. Session, Authentication and Logout

- Frontend SHALL use standards-based OIDC session integration and SHALL not store long-lived secrets in application code.
- On logout, user switch or confirmed session revocation, query caches and sensitive in-memory state SHALL be cleared.
- Idle/session expiry behavior SHALL provide clear re-authentication path.
- Browser storage SHALL not contain raw evidence content, protected-source identity or durable access tokens unless the security architecture explicitly approves the mechanism.
- Multi-tab session expiry SHOULD converge safely without preserving stale privileged UI.

15. Component and Feature Composition

Layer	Examples	State rule
Design system	Button, DataGrid, Badge, ClassificationBanner	No business fetching; presentational/interaction state only.
Domain shared	EvidenceCitation, ProvenanceTrail, FlowClassBadge, EntityBlock	Receives typed domain props; no screen-specific navigation assumptions.
Feature component	EntityMatchCompare, HypothesisMatrix	May own feature queries/mutations and local feature state.
Screen container	SCR-ENT-005, SCR-HYP-001	Coordinates route context, feature components, error/loading states.
App shell	Header, primary nav, session	Owns global navigation/session only.

16. Performance and Rendering Strategy

Concern	Requirement
Route splitting	Feature routes SHOULD be code-split to reduce initial bundle.
Large tables	Use pagination/virtualization where measured data volume requires it; virtualization SHALL preserve keyboard/accessibility.
Graph	Progressive rendering and node limits SHOULD be explicit; large graph failure SHALL degrade to list/filter tools.
Evidence reader	Stream/page-render rather than loading huge files wholly into memory where possible.
Search	Cancel stale requests; cache recent authorized queries briefly.
Re-render control	Memoization SHOULD follow measured cost, not blanket use.
Bundle budget	Critical route bundles SHOULD be monitored in CI with agreed thresholds.

17. Accessibility Architecture

- Route changes SHALL move focus/announce context appropriately.
- Loading, error, save success and validation states SHALL be programmatically perceivable.
- Graph/timeline/value-flow views SHALL have an accessible list/table alternative that references the same canonical objects.

- Keyboard focus SHALL remain recoverable after modal/drawer close and after dynamic mutations.
- Reduced-motion preference SHALL apply to graph transitions and other animations.
- Virtualized tables SHALL retain semantics and keyboard navigation.

18. Frontend Security Requirements

ID	Requirement
FE-SEC-01	No authorization decision relies solely on hidden/disabled UI.
FE-SEC-02	Sensitive API errors SHALL be rendered using non-disclosing approved states.
FE-SEC-03	No raw evidence/protected-source content in analytics, telemetry breadcrumbs or routine console logs.
FE-SEC-04	External links from evidence SHOULD use safe rel/referrer behavior according to policy.
FE-SEC-05	HTML/markdown/user content SHALL be safely rendered; no unsanitized injection.
FE-SEC-06	File previews SHALL be sandboxed/isolated as appropriate and not execute active content.
FE-SEC-07	Browser storage use for case-sensitive state requires explicit security review.
FE-SEC-08	Logout/user switch clears sensitive caches and feature state.

19. Testing Strategy

Layer	Minimum tests
Design-system component	Storybook stories + interaction + accessibility + visual regression.
Feature component	Behavior, permission variants, loading/error, domain-state semantics.
Data hooks	Query keys, invalidation, mutation error mapping, conflict handling.
Route/screen	Critical happy path + permission denied + stale version + responsive smoke.
End-to-end	Case → evidence → entity → value flow → hypothesis → assessment → review → dissemination.
Security	Direct route guesses, cached-data logout, hidden-action API bypass expectation, non-leaking denied states.

Layer	Minimum tests
Accessibility	Keyboard, focus, announcements, graph/list alternatives.

20. Observability and Telemetry

- Frontend errors SHOULD carry correlation/request IDs when provided by API.
- Operational telemetry SHALL avoid case titles, evidence snippets, protected-source identities and free-text analytical content.
- Performance metrics MAY include route load, API latency categories, rendering time and error rate using non-sensitive identifiers.
- User analytics SHALL follow the privacy specification; no keystroke/session replay over sensitive investigation screens by default.

21. Frontend Configuration and Feature Flags

Config	Rule
API base URL	Build/deployment configuration; no hardcoded production URL.
OIDC config	Environment configuration, public client metadata only.
Feature flags	Server/environment policy is authoritative for high-risk features; client flag alone cannot enable API capability.
Build metadata	Version/commit SHOULD be visible in diagnostics.
Experimental graph/AI features	Disabled by default unless approved deployment policy enables them.

22. Screen-to-Feature Ownership Examples

Screen	Feature owner	Primary state owners
SCR-CASE-003	cases	case server state; route caseId; local panel state
SCR-EVD-003	evidence	evidence server state; reader viewport; extract draft form
SCR-ENT-005	entities	candidate server state; compare UI state; merge mutation
SCR-VAL-001	valueflows	flow server state; filters in URL; canvas selection/layout local
SCR-HYP-001	hypotheses	hypothesis server state; matrix filters local; edits form state

Screen	Feature owner	Primary state owners
SCR-REV-002	reviews/products	product version server state; comment form; approval mutation
SCR-DIS-002	dissemination	authorized export candidates server state; selection form; export mutation

23. Frontend Definition of Done

Area	Done when
Architecture	Feature is in correct module and does not create circular/private cross-feature dependencies.
State ownership	Remote, form, route, visualization and ephemeral state have explicit owners.
Authorization	Denied/changed permissions are handled without data leakage; server remains authoritative.
Concurrency	Editable canonical records handle stale versions safely.
UI semantics	Shared design-system/domain components preserve uncertainty/provenance semantics.
Accessibility	Keyboard, focus and state announcements verified.
Tests	Unit/integration/screen/E2E coverage appropriate to risk; Storybook coverage for shared components.
Security	Sensitive content absent from logs/storage/telemetry beyond approved mechanisms.
Performance	Critical route/render performance meets MVP thresholds.
Traceability	Screen IDs and SRS/feature references included in implementation issue/test metadata where practical.

24. Architecture Decision Baseline

Decision	Baseline
FE-ADR-001	Feature-oriented modular React application; no page-folder monolith.
FE-ADR-002	TanStack Query-equivalent owns server state; no duplicate global canonical store.
FE-ADR-003	Form library owns unsaved form state; durable draft requires explicit server object.

Decision	Baseline
FE-ADR-004	React Router-equivalent owns shareable navigation/filter context.
FE-ADR-005	Visualization controllers own layout/selection; graph layout is not canonical truth.
FE-ADR-006	Shared component package/design system before feature-local duplicates.
FE-ADR-007	Version-aware mutations for canonical editable objects.
FE-ADR-008	No offline-first evidence cache in MVP.

Annex A — State Ownership Decision Tree

Is the value canonical or returned by API?
 YES → Server/query state.
 NO → Is it an unsaved user edit?
 YES → Form state.
 NO → Must it survive/share via URL/back button?
 YES → Router/URL state.
 NO → Is it visualization-only (selection/layout/zoom)?
 YES → Visualization controller state.
 NO → Is it transient UI chrome?
 YES → Local component state.
 NO → Define explicit feature ownership; do not default to global store.

Annex B — Frontend Review Checklist

- No canonical object copied into unversioned global mutable state.
- Query key includes relevant case/context dimensions.
- Logout/user switch clears sensitive caches.
- 403/404 non-disclosing states handled correctly.
- Unknown/null/zero distinctions preserved.
- No optimistic high-impact approval/merge/dissemination.
- Stale-version conflict tested.
- Graph/timeline/value-flow derived state does not mutate canonical objects.
- Keyboard/focus/error/loading states tested.
- No protected-source content in ordinary browser storage or telemetry.